Convolution Lab

1. What are the array indexes you used for xzp(...) and h(...) in the `myconv` code?

   Xzp has the array index of (n – k + L) and the index of h is k.

2. Include a stem plot showing the convolution of the following sequences: x = [1, 2, 3, 4, 5, 6, 7, 8] and h = [1 2 3 4]. Also list the values y[n]=x[n]$*h$[n] in the convolution result.

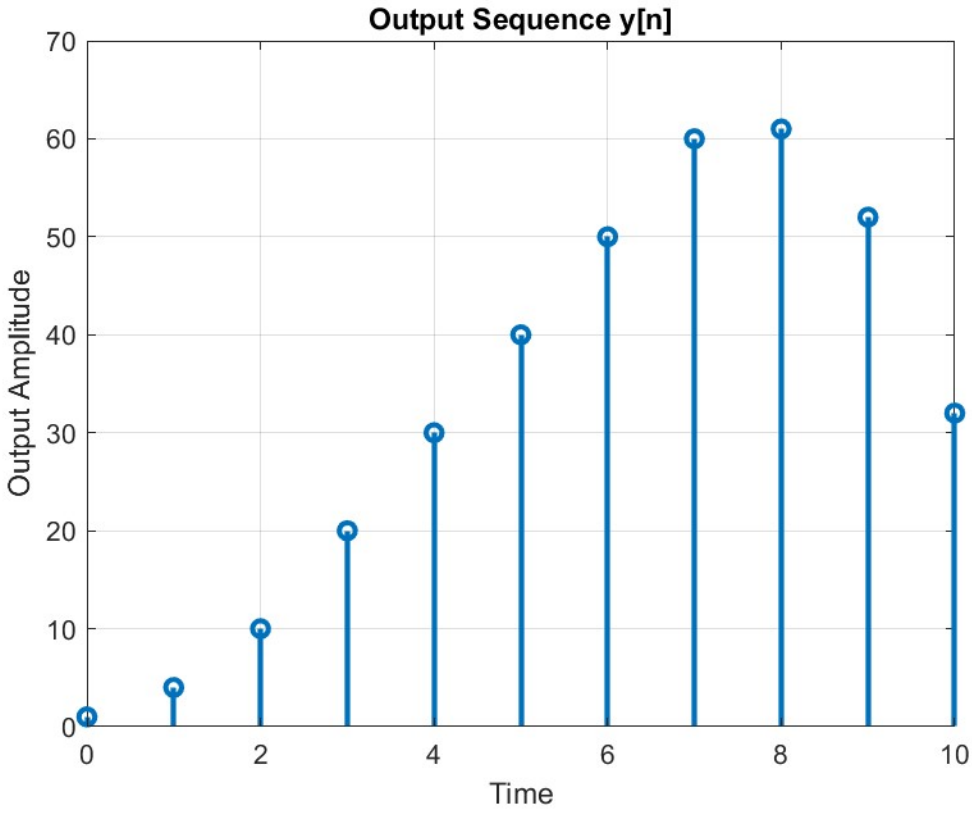


Figure 1: output convolution of x[n]*h[n]

y = [1 4 10 20 30 40 50 60 61 52 32]

3. Plot the impulse response of the filter used in the audio example.

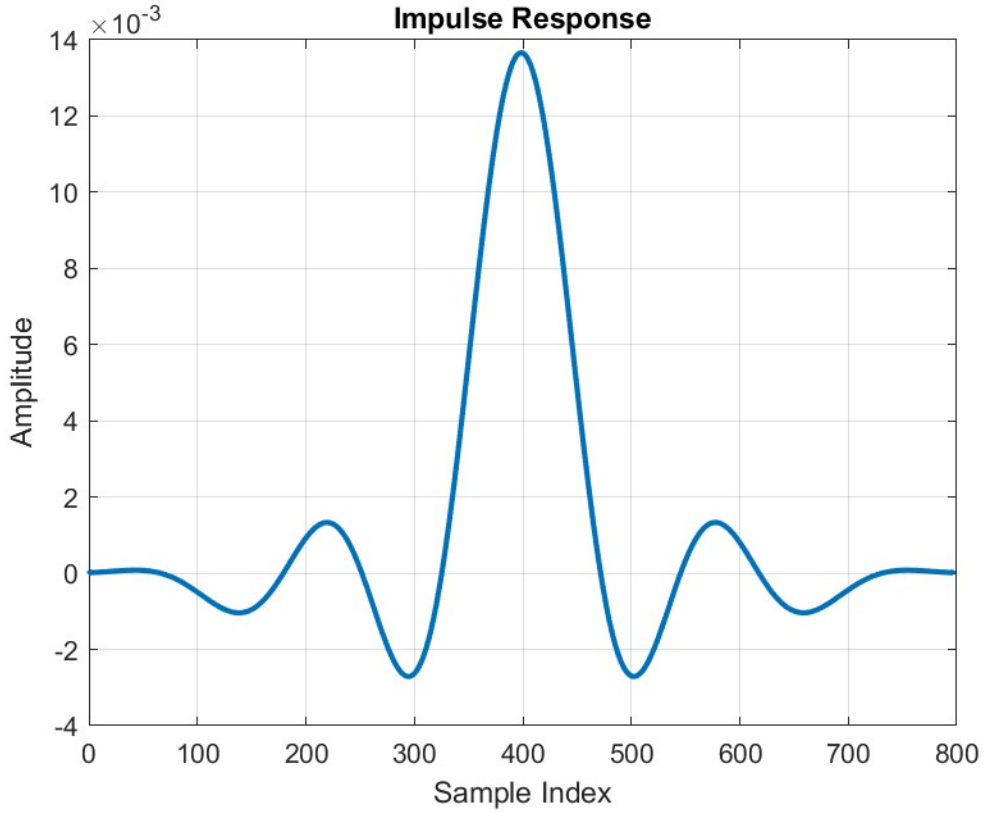


Figure 2: Plot of the impulse response in the time domain

4. Plot the dB-magnitude response of the filter used in the audio example.

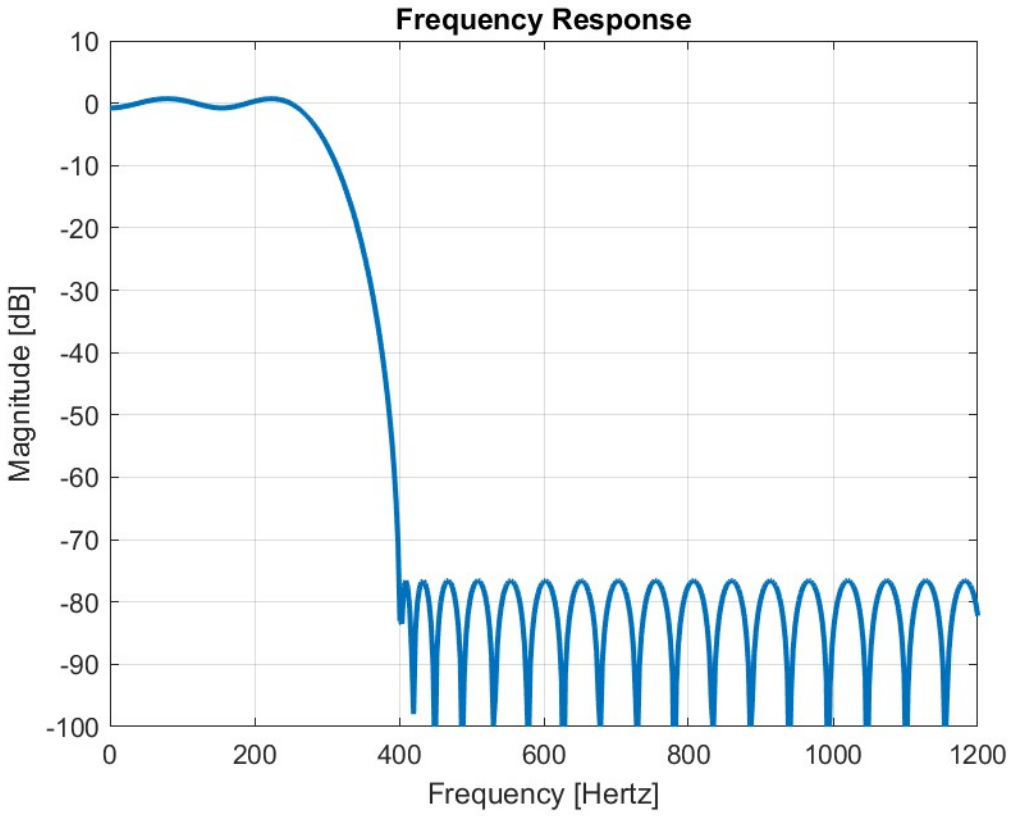


Figure 3: dB-magnitude frequency response of the impulse response

5. Use the magnitude response plot to explain why the filter in the audio example is considered to be a low-pass filter. Describe the features of the passband and the stop band of this filter. What frequencies does this filter "pass" and what frequencies does this filter "stop"?

   The impulse response is considered a low-pass filter because in the frequency response everything past ~400 Hz is attenuated. After the response is passed the frequencies remaining are the low frequencies (0 to 400 Hz).

   The pass and stop bands both feature a ripple effect in the bands. This causes different attenuation for different frequencies. However, for both bands the ripple is not significant enough to impact the audio.

6. What would the response of a high pass filter look like? Make a sketch.

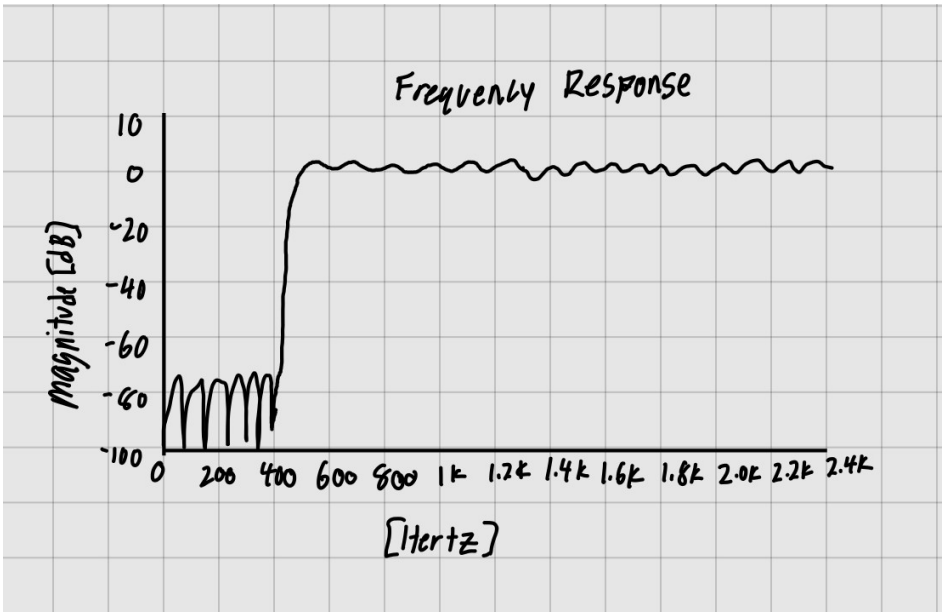


Figure 4: Rough sketch of what a high-pass filter would look like.

7. Plot spectrograms of the audio signals at the filter input and output (2 spectrogram plots).

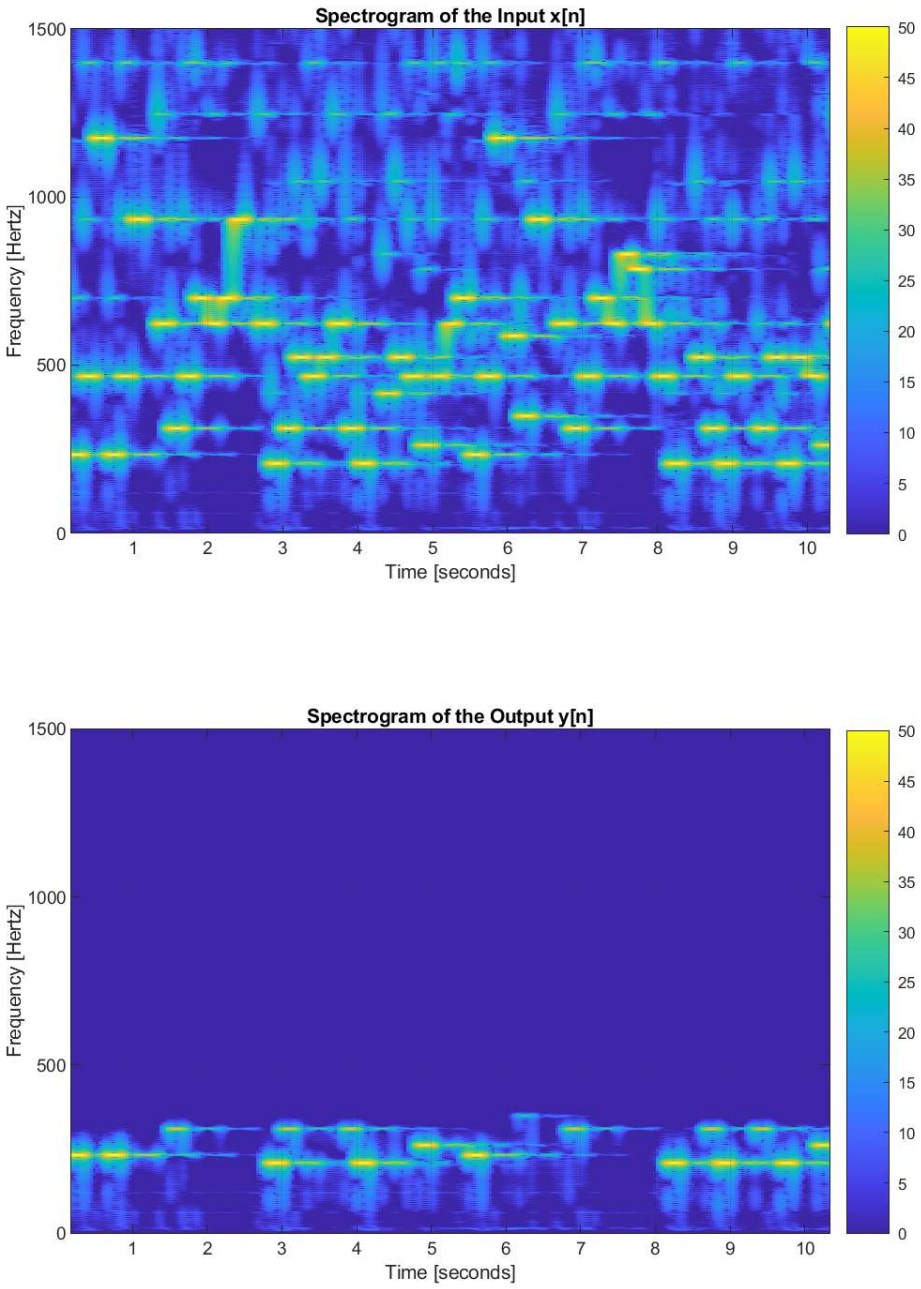


Figure 5: Audio spectrogram of the input and the output after convoluting with the low-pass filter

8. Use the spectrograms to describe the action of this filter on a signal.

In the first spectrograms all the lighter/yellow sections are the frequency and time that certain notes are played. After the signal has gone through the filter everything above 400 Hz is dark blue. This shows that only notes in the lower frequencies are still being played.

9. Plot the filter input and output signals on the same axis (time-domain plot).

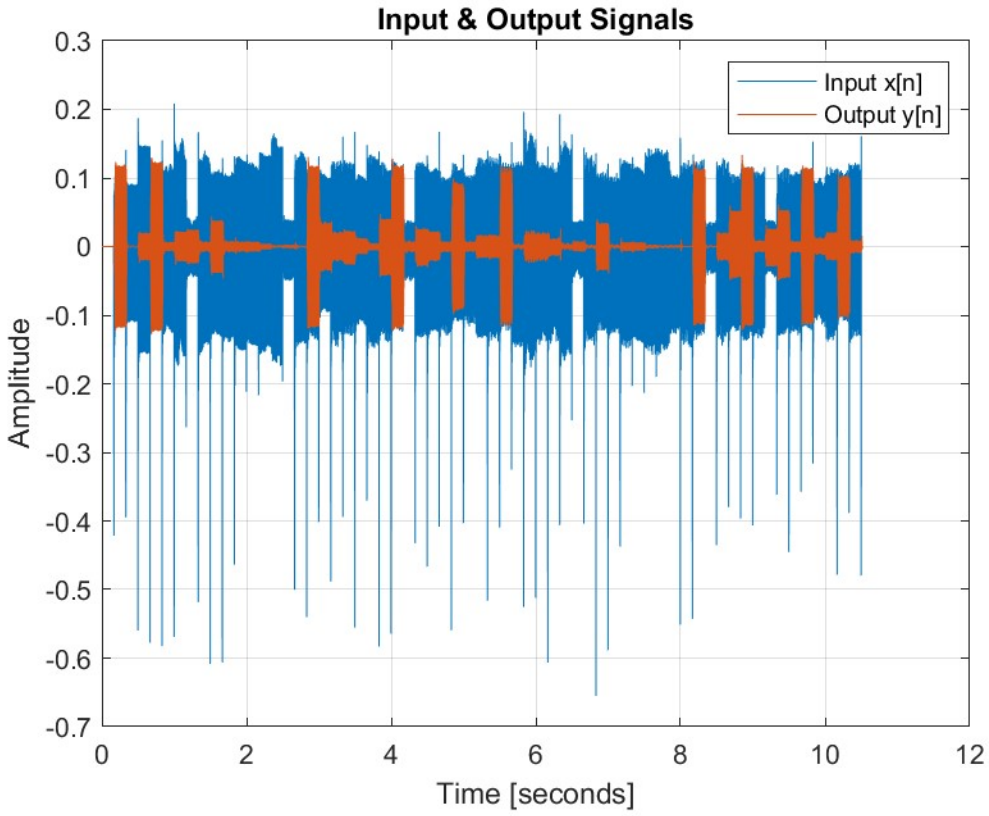


Figure 6: Input audio and output audio from the filter plotted on same axis

10. There are times when the input signal is large and the output signal is essentially zero. See for example the intervals 2-3 seconds and 7-8 seconds in the time-domain plot. Use the filter magnitude response and the spectrograms to explain what is happening in these intervals. Why is the output signal zero there?

    Looking at the spectrogram we can see that the amplitude is dominated by mainly high frequency notes. Therefore, after the low-pass filter those sections the output is essentially 0.

11. Attach your Matlab code.

```
function y = myconv(x, h)
    N = length(x);
    L = length(h);

    K = N + 2*(L-1);
    xzp = zeros(K,1);
    xzp(L:L+N-1) = x;

    M = N+L-1;
    y = zeros(M,1);
    for n=1:M
        for k = 1:L
            y(n) = y(n) + xzp(n - k + L) * h(k);
        end
    end
end
```

```
clear
clc

% Input signal
N = 8;
x = [1 2 3 4 5 6 7 8];


% Impulse response
L = 4;
h = [1 2 3 4];

% Convolve - call custom convolution function
y = myconv(x,h);

% Visualize
M = N+L-1; % Compute length of output
n = [0:M-1]; % Construct a time sequence
stem(n,y,'LineWidth',2); % Plot
xlabel('Time');
ylabel('Output Amplitude');
title('Output Sequence y[n]');
grid on;
shg; % Show handle graphics
print -dpng test_convolution.png
```

```
clear
clc

% Read in impulse response
fid = fopen('lpf_260_400_44100_80dB.bin','rb');
header = fread(fid,5,'int'); % Read in header
h = fread(fid,inf,'float'); % Read in impulse response
L = length(h);
fclose(fid);

% Read in input signal
[x,Fs] = audioread('fireflyintro.wav');
N = length(x);

% Convolve using custom convolution function
y = myconv(x,h);
M = N+L-1;

% Write out the audio signal
audiowrite('fireflyintro_output.wav',y,Fs);

% Visualize the filter impulse response
figure();
plot([0:L-1],h,'LineWidth',2);
xlabel('Sample Index');
ylabel('Amplitude');
title('Impulse Response');
grid on;
shg;
print -dpng impulse_response.png

% Visualize the filter frequency response
NFFT = 2^14;
F = Fs * ([0:NFFT-1]/NFFT - 0.5); % Frequency vector [Hertz]
H = fftshift(fft(h,NFFT)); % Compute Fourier transform
figure();
plot(F,20*log10(abs(H)),'LineWidth',2); % Plot magnitude response in dB
axis([0, 1200, -100, 10]); % Set axis limits
xlabel('Frequency [Hertz]');
ylabel('Magnitude [dB]');
title('Frequency Response');
grid on;
shg;
print -dpng magnitude_response.png

% Visualize signals in time domain
figure();
t = [0:N-1]/Fs; % Time vector [seconds]
plot(t,x); hold on;
t = [0:M-1]/Fs; % Time vector [seconds]
plot(t,y); hold off;
```

```
xlabel('Time [seconds]');
ylabel('Amplitude');
legend('Input x[n]','Output y[n]');
title('Input & Output Signals');
grid on;
shg;
print -dpng audio_timedomain.png

% Plot spectrograms of input and output signals
figure();
subplot(211);
[S,F,T] = spectrogram(x,hamming(NFFT),round(0.9*NFFT),NFFT,Fs);
imagesc(T,F,20*log10(abs(S)),[0 50]);
colorbar;
set(gca,'YDir','normal');
ylim([0 1500]);
xlabel('Time [seconds]');
ylabel('Frequency [Hertz]');
title('Spectrogram of the Input x[n]');
grid on;

subplot(212);
[S,F,T] = spectrogram(y,hamming(NFFT),round(0.9*NFFT),NFFT,Fs);
imagesc(T,F,20*log10(abs(S)),[0 50]);
colorbar;
set(gca,'YDir','normal');
ylim([0 1500]);
xlabel('Time [seconds]');
ylabel('Frequency [Hertz]');
title('Spectrogram of the Output y[n]');
grid on;
shg;
orient tall;
print -dpng audio_spectrograms.png
```